
CSC3209 - Software Architecture and Design Pattern – Practical/Tutorial 3

- 1- Given the following scenario of a project, choose an appropriate architectural pattern and describe the main elements in the selected pattern.

You are asked to develop an information system application. The information is provided to users via several services. Each service provides a different type of information. In this application, the users of the application do not need to know the nature of the service providers and their location to get the required information.

Solution: The broker pattern. The elements are the users(clients), the service providers (servers) and the brokers which are responsible for the coordination between the clients and servers. Using the broker, clients do not need to know about the nature and location of servers to get the provided services.

- 2- Provide an example of an application that uses the layered architectural pattern. For this example:
- Justify your motivations for selecting this pattern.
 - Describe the main elements.
 - Discuss the quality attributes of this pattern: Performance, Ease of development
 - The constraints that should be followed in your example to suit the layered architectural pattern. Justify these constraints specifically in relation to your example.

Solution: To illustrate how the layered architecture works, consider a request from a business user to retrieve customer information for a particular individual as illustrated in Figure 1. The black arrows show the request flowing down to the database to retrieve the customer data, and the red arrows show the response flowing back up to the screen to display the data. In this example, the customer information consists of both customer data and order data (orders placed by the customer).

The customer screen is responsible for accepting the request and displaying the customer information. It does not know where the data is, how it is retrieved, or how many database tables must be queried to get the data. Once the customer screen receives a request to get customer information for a particular individual, it then forwards that request onto the customer delegate module. This module is responsible for knowing which modules in the business layer can process that request and also how to get to that module and what data it needs (the contract). The customer object in the business layer is responsible for aggregating all of the information needed by the business request (in this case to get customer information). This module calls out to the customer dao (data access object) module in the persistence layer to get customer data, and also the order dao module to get order information. These modules in turn execute SQL statements to

retrieve the corresponding data and pass it back up to the customer object in the business layer. Once the customer object receives the data, it aggregates the data and passes that information back up to the customer delegate, which then passes that data to the customer screen to be presented to the user.

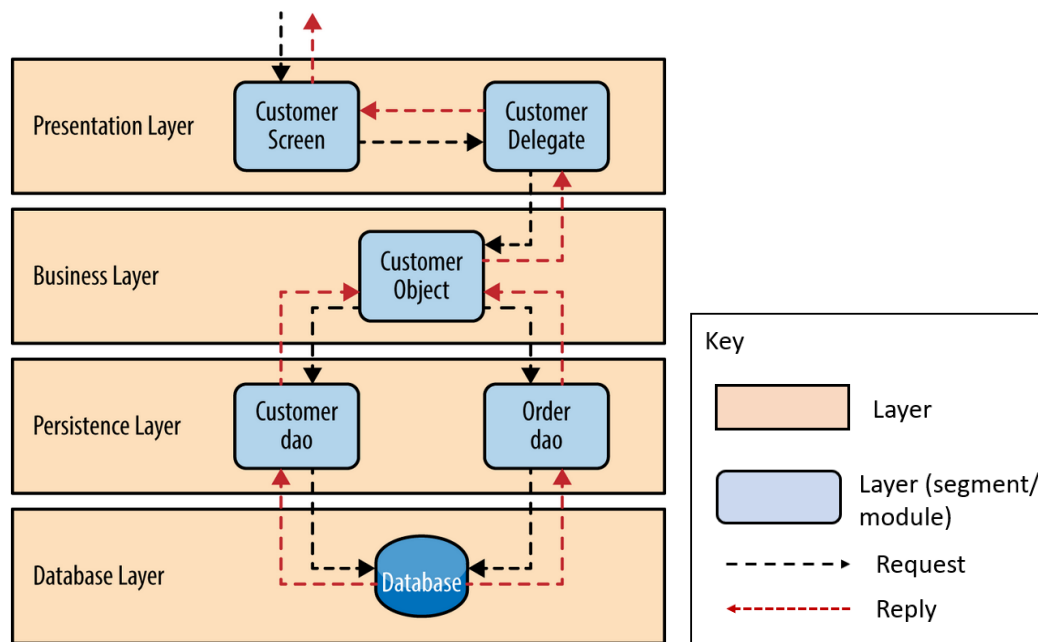


Figure1.

From a technology perspective, there are literally dozens of ways these modules can be implemented. For example, in the Java platform, the customer screen can be a (JSF) Java Server Faces screen coupled with the customer delegate as the managed bean component. The customer object in the business layer can be a local Spring bean or a remote EJB3 bean. The data access objects illustrated in the previous example can be implemented as simple POJO's (Plain Old Java Objects), MyBatis XML Mapper files, or even objects encapsulating raw JDBC calls or Hibernate queries. From a Microsoft platform perspective, the customer screen can be an ASP (active server pages) module using the .NET framework to access C# modules in the business layer, with the customer and order data access modules implemented as ADO (ActiveX Data Objects).

- a- For the motivation: Perform separation of concerns between the different layers.
- b- The main elements are the layers (Presentation, Business, Persistence, Database). These contains the related modules.
- c- The quality attributes:

Performance (Low): While it is true some layered architectures can perform well, the pattern does not lend itself to high-performance applications due to the inefficiencies of having to go through multiple layers of the architecture to fulfill a business request.

Ease of development (High): Ease of development gets a relatively high score, mostly because this pattern is so well known and is not overly complex to implement. Because most companies develop applications by separating skill sets by layers (presentation, business, database), this pattern becomes a natural choice for most business-application development.

- d- The constraints: Check the lecture and relate them to this example. First, every module is allocated to one layer (Customer screen is allocated to the Presentation layer). Second, there are at least two layers. Third, the allowed to use is not circular (a lower layer cannot use a layer above), as this reduces the unnecessary coupling between the layers.
- 3- Provide an example of an application that uses the broker architectural pattern. For this example:
- a. Justify your motivations for selecting this pattern.
 - b. Describe the main elements.
 - c. Discuss the quality attributes of this pattern: Performance, Security, Availability
 - d. The constraints that should be followed in your example to suit the layered architectural pattern. Justify these constraints specifically in relation to your example.

Solution: Check the answer in the first question. The motivation and elements have been stated.

In addition, for the quality attributes:

Performance: Brokers add a layer of indirection and hence latency between clients and servers, and that layer may be a communication bottleneck. However, there is a performance benefit (because the broker pattern makes it easy to assign work to the least busy server).

Security: A broker may be a target for security attacks (so it needs to be hardened appropriately).

Availability: (because the broker pattern makes it easy to replace a failed server with another).